



- ▶ **Lenguajes compilados e interpretados**
- ▶ **Entornos virtuales**
- ▶ **Ipython básico**

Bibliografía

- *Introducción a la programación con Python. secciones 3.2, 3.3 y 3.4*, Andrés Marzal, Isabel Gracia y Pedro García.
- *Computational Physics, cap. 2*, Mark Newman.

Scripts (.py)

Hasta el momento, hemos trabajado en el entorno interactivo de Python, llamado IPython. Sin embargo, esto limita mucho nuestra capacidad de trabajo a la hora de programar. Por esta razón, es mejor utilizar editores de texto y construir nuestro programa allí. Existe una convención con la extensión de los ficheros (caracteres que suceden al punto), la convención es usar `.py`. Por ejemplo, `primer_progama.py`.

En algunos sistemas operativos las extensiones suelen ocultarse por defecto, pero se recomienda no hacerlo. Siempre es importante nombrar los archivos con su extensión para que tanto el editor como el sistema operativo identifiquen correctamente cómo interpretarlos.

`primer_programa.py`

```
1 """ Nuestro primer programa en Python.
2 Pregunta algunos datos personales"""
3
4 # con el comando print(""), imprimimos un mensaje en cosa \n salta la linea
5 print("¡Hola, espero que te encuentres muy bien!")
6
7 #El comando input se utiliza para ingresar información.
8 #Por defecto la variable asignada es un string
9 nombre = input("¿Cuál es tu nombre?\n")
10
11 print(f"-----")
12 # Uso de f-strings para la salida
13 print(f"¡Mucho gusto, {nombre}!")
14
15 # Conversión de datos.
16 # int() convierte la entrada a un entero
```

```
17 print(f"-----")
18 edad= int(input("¿Cuántos años tienes?\n"))
19 print(f"-----")
20 # Hacemos operaciones con ese entero
21 edad_segundos = edad * 365 * 24 * 3600 #calcula edad en segundos
22 edad_minutos = edad * 365 * 24 * 60 #calcula edad en minutos
23 futura_edad = edad + 5 # edad dentro de 5 años
24 print(f"Tu edad en minutos es {edad_minutos}. En notación científica es {
    edad_minutos:.2e} minutos")
25 print(f"Tu edad en segundos es {edad_segundos}. En notación científica es {
    edad_segundos:.2e} segundos")
26 print(f"Si todo sale bien, te graduarías de Física a los {futura_edad} años.
    ")
27 print("-----")
28
29 #EDAD DEL UNIVERSO
30 # Edad del universo aproximada en años (13 800 millones de años)
31 edad_universo = 13.8e9 # Notación científica en Python para float
32 edad_universo_segundos = edad_universo * 365.25 * 24 * 3600 # edad universo
    en segundos
33
34 # Calculamos el porcentaje que representa la edad de entrada respecto a la
    del Universo
35 porcentaje = (edad / edad_universo) * 100
36
37 print(f"Por otro lado, la edad del universo es de {edad_universo:.2e} años,"
    )
38 print(f"lo cual equivale a aproximadamente: {edad_universo_segundos:.2e}
    segundos.")
39 print(f"Eso significa que has vivido aproximadamente el {porcentaje:.10f}%
    de la historia del universo.")
40 print("-----")
41
42 # EDAD DE LA TIERRA
43 edad_tierra = 4.54e9 # Edad de la Tierra aproximada en años (4540 millones
    de años)
44 edad_tierra_segundos = edad_tierra * 365.25 * 24 * 3600 # Edad de la Tierra
    en segundos
45
46 # Calculamos el porcentaje que representa la edad de entrada respecto a la
    Tierra
47 porcentaje_tierra = (edad / edad_tierra) * 100
48
49 print(f"Además, la edad de la Tierra es de {edad_tierra:.2e} años ({
    edad_tierra_segundos:.2e} segundos).")
50 print(f"Has presenciado aproximadamente el {porcentaje_tierra:.8f}% de la
    existencia de nuestro planeta.")
51 print("-----")
52
53 # 3. APARICIÓN DEL HOMO SAPIENS
```

```
54 edad_humanidad = 300000.0 # 300 000 años aproximadamente
55 edad_humanidad_segundos = edad_humanidad * 365.25 * 24 * 3600
56
57 porcentaje_humano_universo = (edad_humanidad / edad_universo) * 100
58 porcentaje_humano_tierra = (edad_humanidad / edad_tierra) * 100
59
60 print(f"En perspectiva evolutiva, los primeros Homo sapiens aparecieron hace
    {edad_humanidad} años.")
61 print(f"En segundos, la humanidad lleva existiendo aproximadamente {
    edad_humanidad_segundos:.2e} s (~10~13 s).")
62 print(f"- Nuestra especie solo representa el {porcentaje_humano_tierra:.4f}%
    de la historia de la Tierra.")
63 print(f"- Respecto al universo, la humanidad solo abarca el {
    porcentaje_humano_universo:.5f}% de su historia.")
64 print("-----")
65 print("-----")
```

Este script anterior introduce los conceptos fundamentales de interacción con la consola, manejo de variables, conversión de tipos y formato de datos numéricos en Python.

1. Entrada y Salida (input y print): La función `print()` envía datos a la salida estándar (consola). La secuencia de escape `\n` genera un salto de línea dentro de la cadena.

La función `input()` detiene la ejecución del programa y espera a que se ingrese información desde el teclado. **Importante:** Toda entrada capturada por `input()` se almacena internamente como una cadena de texto (`str`), independientemente de si el usuario ingresó números.

2. Conversión de Tipos (Casting): Para realizar operaciones aritméticas con la variable `edad`, es necesario convertir el texto recibido a un tipo numérico computable. En este caso se utiliza `int()` para convertir la entrada en un número entero:

```
edad = int(input("..."))
```

Si intentáramos multiplicar directamente el resultado de `input()` sin convertirlo, Python duplicaría la cadena de texto en lugar de realizar la operación matemática.

3. Tipos Flotantes (float) y Notación Científica: Para magnitudes grandes como la edad del universo ($13,8 \times 10^9$ años), Python utiliza notación científica con la letra `e`:

```
edad_universo = 13.8e9  $\equiv 1,38 \times 10^{10}$ 
```

Estas variables pertenecen al tipo de dato de punto flotante (`float`).

4. Formato de Cadenas (f-strings): Las cadenas formateadas permitidas mediante `f"..."` integran variables y expresiones dentro del texto entre llaves `{}`. Además, admiten especificadores de formato para controlar la precisión:

- `{variable:.2e}`: Muestra el valor en notación científica con 2 cifras decimales.
- `{variable:.10f}`: Muestra el valor en punto flotante tradicional fijando 10 dígitos decimales (ideal para porcentajes astronómicos pequeños).

5. Uso de Comentarios (#): Las líneas que comienzan con el símbolo numeral # son comentarios de una sola línea. El intérprete de Python ignora por completo estas líneas durante la ejecución. Se utilizan para:

- Documentar y explicar la lógica del código a otros programadores (o a uno mismo en el futuro).
- Aclarar las unidades físicas empleadas en los cálculos (e.g., conversión de años a segundos).
- Inhabilitar temporalmente líneas de código durante el desarrollo y depuración (*debugging*).

Notebooks de Jupyter (.ipynb)

Hasta el momento construimos nuestro programa como un script (`primer_programa.py`), el cual se ejecuta secuencialmente de principio a fin en la terminal. Sin embargo, en el ámbito de la investigación y la física computacional, también es común utilizar **Jupyter Notebooks**.

Un notebook es un entorno interactivo basado en la web que combina:

- **Celdas de texto de marcas (Markdown):** Para documentar la teoría, incluir ecuaciones en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, explicaciones y gráficos.
- **Celdas de código:** Para escribir y ejecutar fragmentos independientes de Python, conservando el estado de las variables en memoria.

A diferencia de un script tradicional, en un notebook podemos fragmentar el programa en celdas lógicas y ejecutar cada bloque por separado sin necesidad de reiniciar todo el programa.

Por ejemplo, en el archivo `volumen.ipynb` se calcula el volumen y el área de una esfera a partir del radio ingresado:

$$V = \frac{4}{3}\pi R^3 ; \quad A = 4\pi R^2$$

[Celda 1 - Markdown]

Cálculo del Volumen de una Esfera

En este notebook le pedimos al usuario/a que ingrese el radio R en metros de una esfera. Con ese dato calcula el volumen: $V = \frac{4\pi R^3}{3}$ y el área $A = 4\pi R^2$

[Celda 2 - Código]

```
1 import math # Importamos el modulo matemático estándar de Python
2
3 print("=== CÁLCULO DEL VOLUMEN DE UNA ESFERA ===")
4
5 # Solicitamos el radio al usuario.
6 # Usamos float() porque el radio puede ser un número decimal (e.g., 2.5)
7 radio = float(input("Ingresa el radio de la esfera R (en metros):\n"))
8
9 print(f"\nRadio ingresado: {radio} m")
```

[Celda 3 - Markdown]

$V = \frac{4\pi R^3}{3}$ y el área $A = 4\pi R^2$

[Celda 4 - Código]

```

1 V = 4./3 * math.pi* radio**3 # calculamos el volumen
2 A = 4 * math.pi* radio**2 # calculamos el area
3 print("-----")
4 print(f"El volumen de la esfera es: {V:.4f} m^3")
5 print(f"En notación científica:      {V:.3e} m^3")
6 print(f"El area de la esfera es: {A:.4f} m^2")
7 print(f"En notación científica:      {A:.3e} m^2")
8
9 print("-----")

```

Note los siguientes detalles:

- 1. Uso de `float(input())`:** A diferencia del programa anterior donde usábamos `int()` para una edad contable entera, el radio es una cantidad física continua. Por ello, empleamos `float()` para permitir números decimales (e.g., 1.5) y notación científica en la entrada (e.g., 6.371e6 para el radio de la Tierra).
- 2. Importación del módulo `math`:** Python no incluye la constante π de forma nativa en su ámbito global. La instrucción `import math` carga el módulo matemático estándar, permitiendo acceder a `math.pi` con alta precisión (15 dígitos decimales).
- 3. Operador de Potencia (`**`):** En Python, la exponenciación se realiza con el operador doble asterisco `**` (e.g., `radio ** 3`).

Ventajas y advertencias al usar Notebooks

Persistencia de variables en memoria: Una vez que ejecutas la **Celda 2**, las variables `nombre` y `edad` quedan almacenadas en la sesión activa del Kernel. Puedes usarlas en las celdas 4 y 6 sin volver a pedir las al usuario.

El orden de ejecución importa: A diferencia de un script secuencial, en un notebook las celdas se pueden ejecutar fuera de orden. El número a la izquierda de la celda (e.g., In [1] :) indica el orden real en que el Kernel procesó el código, no la posición visual en la página.

Atención con `input()` en Notebooks: Cuando ejecutas una celda con `input()` en VS Code o Jupyter-Lab, aparecerá una casilla de texto especial en la parte superior del editor o justo debajo de la celda. Debes ingresar el valor y presionar Enter; de lo contrario, la celda se quedará en estado de espera infinito y no permitirá ejecutar otras celdas.

Problemas:

1. Elabore un programa que muestre en pantalla el enunciado de la ley de gravitación Universal de Newton.
2. Elabore un programa que cuando se ejecute muestre en pantalla un párrafo de su canción preferida.
3. **Conversión de coordenadas polares a cartesianas:** Suponga que la posición de un punto en

el espacio bidimensional nos viene dada en coordenadas polares r, θ y queremos convertirla a coordenadas cartesianas $x = r \cos \theta$, $y = r \sin \theta$. Elabore un programa que:

- Pida ingresar los valores de r y θ . Cuidado, para evaluar θ , recuerde que debe estar en radianes.
- Convierta esos valores a coordenadas cartesianas utilizando las fórmulas estándar:

$$x = r \cos \theta, \quad y = r \sin \theta.$$

- Imprima los resultados.

4. **Conversión de coordenadas cartesianas a polares:** Construya un programa que realice la operación contraria a la anterior. Es decir, que pida un punto en coordenadas cartesianas y lo devuelva en polares. Recuerde que $r^2 = x^2 + y^2$, $\tan \theta = \frac{y}{x}$.

5. **Altitud de un satélite:** Cuando se lanza un satélite a una órbita circular alrededor de la Tierra de modo que orbite el planeta una vez cada T segundos, la altura alcanzada por el satélite sobre la superficie de la Tierra es

$$h = \left(\frac{GMT^2}{4\pi^2} \right)^{1/3} - R,$$

donde $G = 6,67 \times 10^{-11} \text{ m}^3\text{kg}^{-1}\text{s}^{-2}$ es la constante de gravitación universal de Newton, $M = 5,97 \times 10^{24} \text{ kg}$ es la masa de la Tierra y $R = 6371 \text{ km}$ es su radio.

- Escriba un programa que solicite ingresar el valor deseado de T y luego calcule e imprima la altitud correcta en metros.
- Utilice su programa para calcular las altitudes de satélites que orbitan la Tierra una vez al día (la llamada órbita “geoestacionaria” o “geosíncrona”), una vez cada 90 minutos y una vez cada 45 minutos.

6. **Velocidad de Escape:** La velocidad de escape (v_e) (rapidez mínima que debe alcanzar un cuerpo para escapar del campo gravitacional) para escapar del pozo gravitacional generado por una masa esférica M de radio R es:

$$v_e = \sqrt{\frac{2GM}{R}}$$

donde $G = 6,674 \times 10^{-11} \text{ m}^3\text{kg}^{-1}\text{s}^{-2}$ es la constante de gravitación universal. Construya un script con las siguientes instrucciones:

- Definir la constante G utilizando la sintaxis de notación científica en Python ($6.674\text{e-}11$).
- Solicitar la masa M (en kg) y el radio R (en m) del objeto astronómico.
- Calcular v_e
- Presentar la velocidad de escape calculada tanto en m/s como en km/s.
- Casos de estudio (Pruebas de validación):

- **Planeta Tierra:** $M = 5,972 \times 10^{24} \text{ kg}$ ($5.972\text{e}24$), $R = 6,371 \times 10^6 \text{ m}$ ($6.371\text{e}6$).
Resultado esperado: $\approx 11,18 \text{ km/s}$.

- **Estrella de Neutrones:** $M = 2,8 \times 10^{30} \text{ kg}$ ($2.8\text{e}30$), $R = 1,2 \times 10^4 \text{ m}$ ($1.2\text{e}4$).

7. **Órbitas planetarias:** La órbita en el espacio de un cuerpo alrededor de otro, como la de un planeta alrededor del Sol, no tiene por qué ser circular. En general adopta la forma de una elipse, con el cuerpo estando a veces más cerca y a veces más lejos. Si se le da la distancia ℓ_1 de máximo acercamiento que hace un planeta al Sol, también llamada su *perihelio*, y su velocidad lineal v_1 en el perihelio, entonces cualquier otra propiedad de la órbita se puede calcular a partir de estas dos de la siguiente manera:

- a) La segunda ley de Kepler nos dice que la distancia ℓ_2 y la velocidad v_2 del planeta en su punto más distante, u *afelio*, satisfacen $\ell_2 v_2 = \ell_1 v_1$. Al mismo tiempo, la energía total, cinética más gravitacional, de un planeta con velocidad v y distancia r al Sol está dada por

$$E = \frac{1}{2}mv^2 - G\frac{mM}{r},$$

donde m es la masa del planeta, $M = 1,9891 \times 10^{30} \text{ kg}$ es la masa del Sol y $G = 6,6738 \times 10^{-11} \text{ m}^3\text{kg}^{-1}\text{s}^{-2}$ es la constante de gravitación de Newton. Dado que la energía debe conservarse, demuestre que v_2 es la raíz menor de la ecuación cuadrática

$$v_2^2 - \frac{2GM}{v_1 \ell_1} v_2 - \left[v_1^2 - \frac{2GM}{\ell_1} \right] = 0.$$

Una vez que tenemos v_2 , podemos calcular ℓ_2 usando la relación $\ell_2 = \ell_1 v_1 / v_2$.

- b) Dados los valores de v_1 , ℓ_1 y ℓ_2 , otros parámetros de la órbita vienen dados por fórmulas simples que se pueden derivar de las leyes de Kepler y del hecho de que la órbita es una elipse:

$$\text{Semieje mayor: } a = \frac{1}{2}(\ell_1 + \ell_2),$$

$$\text{Semieje menor: } b = \sqrt{\ell_1 \ell_2},$$

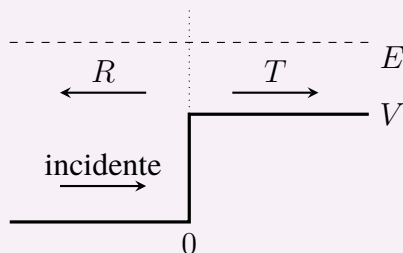
$$\text{Período orbital: } T = \frac{2\pi ab}{\ell_1 v_1},$$

$$\text{Excentricidad orbital: } e = \frac{\ell_2 - \ell_1}{\ell_2 + \ell_1}.$$

Escriba un programa que solicite ingresar la distancia mínima al Sol y la velocidad en el perihelio, y luego calcule e imprima las cantidades ℓ_2 , v_2 , T y e .

- c) Pruebe su programa haciendo que calcule las propiedades de las órbitas de la Tierra (para la cual $\ell_1 = 1,4710 \times 10^{11} \text{ m}$ y $v_1 = 3,0287 \times 10^4 \text{ m s}^{-1}$) y del cometa Halley ($\ell_1 = 8,7830 \times 10^{10} \text{ m}$ y $v_1 = 5,4529 \times 10^4 \text{ m s}^{-1}$). Entre otras cosas, debería encontrar que el período orbital de la Tierra es de un año y el del cometa Halley es de unos 76 años.

8. **(Opcional) Escalón de potencial cuántico:** Un problema bien conocido de la mecánica cuántica involucra una partícula de masa m que se encuentra con un escalón de potencial unidimensional:



La partícula, con energía cinética inicial E y vector de onda $k_1 = \sqrt{2mE}/\hbar$, ingresa desde la izquierda y encuentra un salto repentino en la energía potencial de altura V en la posición $x = 0$. Al resolver la ecuación de Schrödinger, se puede demostrar que cuando $E > V$ la partícula puede: (a) pasar el escalón, en cuyo caso tiene una energía cinética menor dada por $E - V$ al otro lado y un vector de onda correspondientemente menor $k_2 = \sqrt{2m(E - V)}/\hbar$, o (b) reflejarse, conservando toda su energía cinética y un vector de onda inalterado, pero moviéndose en la dirección opuesta. Las probabilidades T y R de transmisión y reflexión están dadas por:

$$T = \frac{4k_1k_2}{(k_1 + k_2)^2}, \quad R = \left(\frac{k_1 - k_2}{k_1 + k_2} \right)^2.$$

Suponga que tenemos una partícula con masa igual a la masa del electrón $m = 9,11 \times 10^{-31}$ kg y energía 10 eV que se encuentra con un escalón de potencial de altura 9 eV. Escriba un programa en Python para calcular e imprimir las probabilidades de transmisión y reflexión utilizando las fórmulas anteriores. $\hbar = \frac{h}{2\pi}$ es la constante reducida de Planck, cuyo valor es $\hbar \approx 1,0545718 \times 10^{-34}$ J · s $\approx 6,582119 \times 10^{-16}$ eV · s.